

Jerith Coin (JER)

A Social Proof-of-Work Currency for the AI Era

The native asset of JerithChain — mined through conversation with the Jerith AI agent, secured by SHA-256 proof-of-work, and settled entirely on-chain.

Chain: JerithChain **Ticker:** JER **Consensus:** Nakamoto PoW **Max supply:** 1,000,000,000 JER

Status: Live · v1.0 · October 2026

CONTENTS

1. [Abstract](#)
2. [Motivation](#)
3. [Architecture](#)
4. [Consensus & Block Structure](#)
5. [Tokenomics](#)
6. [Mining Model](#)
7. [Accounts, Wallets & Security](#)
8. [Network Topology & Synchronization](#)
9. [Transparency & Privacy](#)
10. [Security Discussion](#)
11. [Implementation Status](#)
12. [Roadmap](#)
13. [Conclusion](#)
14. [References & Resources](#)

1 • Abstract

Jerith Coin (JER) is a proof-of-work digital asset whose issuance is bound to a social activity: conversation with an AI agent. Every message sent to **Jerith** — a self-hosted assistant operating on the Discord platform — carries a small probability of minting a block reward for the sender, and users may deliberately mine for the full block reward. The chain itself is deliberately conventional: SHA-256 *Nakamoto* proof-of-work, Ed25519-signed transactions, an account/balance ledger derived exclusively from applied transactions, and a difficulty retargeting rule bounded in both directions. The design goal is a community currency with an issuance curve driven by engagement rather than speculation, while inheriting the well-understood security properties of a single-chain PoW system.

JerithChain is fully implemented and running. The reference node, wallet tooling, gRPC-based block synchronization, a public explorer, and a 23-case test suite are open source under the MIT license.

2 • Motivation

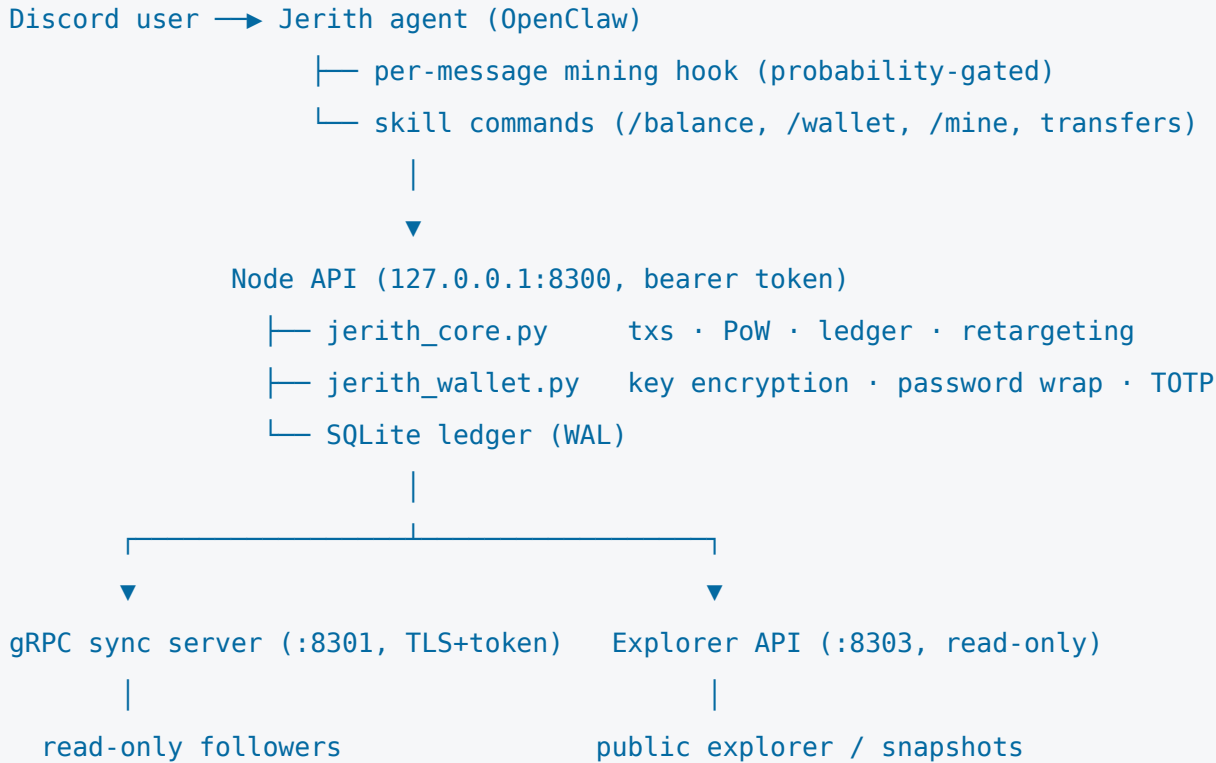
Traditional mineable coins reward capital (ASICs) or idle hardware (GPUs). Proof-of-stake systems reward capital directly. Both mechanisms concentrate issuance among their largest participants and give non-technical users no role in securing the network.

Jerith inverts the premise: the scarce resource being spent is **meaningful participation** in an AI-assisted creative community. The chain uses proof-of-work for security and final settlement, but the *right to submit* proof-of-work is metered socially — through rate limits tied to a person's Discord identity. This yields three properties:

- **Fair engagement curve.** Rewards track conversation and contribution, not hashrate. A casual user on modest hardware earns alongside the operator's own fleet.
- **Sybil resistance by identity.** One wallet per verified platform identity, with platform-side moderation as an additional lever unavailable to permissionless chains.
- **Honest plumbing.** The monetary core avoids experimental cryptography. Consensus, signatures, and validation are deliberately boring and auditable; innovation lives in the issuance policy, not the trust model.

3 · Architecture

JerithChain is a single-chain, UTXO-free (account/balance) ledger written in Python. A full node is a single service exposing a JSON/HTTP API on loopback; block production, validation, and storage are performed in one process with serialized writes.



3.1 Data model

- **Transaction.** {kind, sender, recipient, amount, fee, nonce, timestamp, memo, vk, signature}. `kind ∈ {spend, mine, premine}`; only `spend` may be user-submitted. The verify key `vk` must hash-derive to `sender`, binding the account identity to the signing key.
- **Block.** {height, prev_hash, timestamp, txs[], miner, reward, nonce}; the block hash is `SHA-256` over the canonical JSON serialization of these fields.
- **State.** Balances and nonces are stored per address and recomputed only from applied transactions; the ledger is the sole source of truth.

4 • Consensus & Block Structure

4.1 Proof-of-work

A block is valid when its hash has at least `D` leading zero bits, where `D` is the current difficulty (`D = 18` at genesis, $\approx 2^{18}$ hash attempts per block). Miners increment `nonce` and re-hash; there is no memory hardness, by design.

4.2 Difficulty retargeting

Every `R = 20` blocks, difficulty is recomputed from the measured wall-clock span of the prior window versus the 1,200-second target (20×60 s):

```
ratio    = clamp( expected_dt / actual_dt , 1/4 , 4 )
D_new    = clamp( D_prev + round(log2 ratio) , 8 , 30 )
```

The $4\times$ per-window clamp and 8–30 bit absolute bounds prevent difficulty shocks from a minority miner joining or leaving; the log-linear bit adjustment mirrors Bitcoin's bowl-shaped retarget behavior at a much shorter horizon. Retarget inputs are the *stored* block timestamps of the prior window, so recomputation is deterministic from the ledger and nodes never disagree.

4.3 Validation rules

- PoW difficulty met, and `prev_hash` equals the tip's hash.
- Reward txs (`mine` , `premine`) must be chain-minted: they carry no user signature path and are rejected if submitted by users.
- Spend txs require a valid Ed25519 signature over the canonical payload, a sender/vk match, a strictly sequential nonce, and sufficient balance including the burned fee.
- Rewards are clamped to the remaining emission; supply can never exceed the cap even under a consensus bug in reward scheduling.
- Fee burns are applied at the same time as the transfer — total supply only ever decreases via burns.

5 • Tokenomics

PARAMETER	VALUE	NOTES
Max supply	1,000,000,000 JER	Hard cap, enforced at validation
Genesis premine	200,000,000 JER	20% — founder allocation, publicly visible at height 0
Block reward	50 JER	Halves every 210,000 blocks (~145 days at target)
Target block time	60 s	Retarget window: 20 blocks
Transfer fee	0.1 JER	Flat, burned on every spend
Unit	μJ internally	1 JER = 1,000,000 μJ; integer math only
Deflationary pressure	Fee burns	Net emission = rewards – burns

The 20% premine funds long-term development, liquidity, and community grants; it is fully transparent on-chain from genesis and its movements are auditable through the public explorer. Emission to year one at target block time is $\approx$ 17.1M JER (~1.7% of cap) — 210,000 blocks at 50 JER, then 210,000 at 25 JER, then 105,600 at 12.5 JER — tapering with each halving.

6 • Mining Model

JerithChain separates *who may mine* (social, metered) from *what secures the chain* (proof-of-work, permissionless to verify). Two paths exist:

6.1 Passive mining

Each inbound message to Jerith is independently eligible at a 12% probability; a win mints a block capped at 2 JER for the sender. Cooldowns (300 s success / 180 s failure) apply per identity and are enforced server-side inside the node's serialized write path, making reward races structurally impossible. Message-level deduplication makes replay of delivery events idempotent.

6.2 Explicit mining

A user may deliberately mine for the full block reward (50 JER at current height). The node performs the hash search itself — mining is **not** compute-gated for the user — and the same cooldowns apply. This is deliberately egalitarian: users do not need GPUs, and the operator gains no mining advantage beyond what any participant has.

6.3 Opt-out redirection

Participation is opt-in by default. An opted-out identity's mining rights still consume the global issuance schedule, but rewards are credited to the community pool instead of the user's wallet. The user's balance and history are untouched; re-enabling restores direct crediting. This keeps the emission curve independent of individual participation choices — total issuance remains a function of community activity, not of who happens to be opted in.

7 • Accounts, Wallets & Security

7.1 Identity binding

Addresses are `JER || SHA-256("jerith-addr" || vk)[0:32].upper()` — 35 characters, checksummed by construction (a typo does not hash-match any registered account). One wallet is bound to one platform identity; rebinding requires proving knowledge of the wallet's private key.

7.2 Key custody

Ed25519 secret keys (32-byte seeds) are stored doubly encrypted: AES-256-GCM under a node master key (OS-protected file, 0600) *and* a second scrypt-wrapped envelope under the user's password. Sovereignty operations — key export, identity rebind, TOTP enrollment changes, and spends above the auto-sign policy — require the password plus a valid RFC-6238 TOTP code, enforced server-side with attempt lockouts and audit logging.

7.3 Auto-sign policy

Spends at or below 100 JER per transaction and 2,000 JER per rolling day are signed automatically. Above either bound, user re-authorization (password + TOTP) is required. The policy is enforced in the node, not the agent, so a compromised conversational layer cannot drain a wallet beyond the daily bound without explicit 2FA.

7.4 Key export & portability

Users may export their secret key (rate-limited to once per hour, 2FA-gated, private-channel delivery only) and later rebind the same wallet to a new platform identity. The address, balance, and history are invariant under rebind; the identity is metadata, not the account.

8 • Network Topology & Synchronization

Today JerithChain operates as an operator-rooted star: the founding node authors blocks, and any number of **read-only followers** mirror the ledger over gRPC streams secured with TLS and a bearer token. Followers verify every received block — PoW, linkage, signature validity, reward schedule, supply cap — before applying it, and re-sync on divergence by discarding their tail. A public chain snapshot (full recent history plus address balances) is published every five minutes for lightweight verification and explorer use.

This topology is honest about its stage: it is a **federated single-authority chain** with independent verification, not yet a decentralized validator set. The roadmap (§12) sequences the path to multi-authority block production without changing the validation rules that followers already enforce.

9 · Transparency & Privacy

- **Pseudonymity.** Addresses are not directly linked to platform identities in chain data; reward memos record the platform ID that earned a block, by design, making social issuance publicly auditable.
- **No hidden state.** There are no private side-ledgers. Every balance change — including the genesis premine — is a visible transaction.
- **Minimal personal data.** Nodes store platform IDs and public keys only; no emails, no profile data. Wallet key material is encrypted at rest and never leaves the node except through the explicit, 2FA-gated export flow.
- **Regulatory posture.** Jerith Coin makes no privacy guarantees beyond pseudonymity; users should treat chain activity as public.

10 · Security Discussion

10.1 Consensus-level

- **51% attack.** As a single-authority chain with independent verifiability, block authorship is trusted to the operator; followers and third parties can verify but not fork. Multi-authority mining (§12) is the mitigation path; the PoW substrate requires no changes to get there.
- **Difficulty manipulation.** Timestamp windows are clamped (4×) and bounded (8–30 bits); an operator inserting favorable timestamps can shift issuance timing but not total supply, which is capped at validation.
- **Replay.** Per-account sequential nonces make replay impossible; message-level delivery deduplication adds an application-layer guarantee.

10.2 Application-level

- **Agent compromise.** The conversational agent holds no keys; it calls the node's token-authenticated API, which enforces auto-sign caps and 2FA gates independent of the agent.
- **Node compromise.** Master-key theft alone cannot export user keys (second script envelope requires user passwords); per-user TOTP further gates sovereignty operations. Failures are rate-limited and audited.
- **Economic spam.** Burned fees price ledger writes; social rate limits price mining attempts.

11 • Implementation Status

JerithChain v1.0 is live. The reference implementation is ~2,600 lines of dependency-light Python (PyNaCl, cryptography, FastAPI, grpcio) with SQLite storage.

- **Tested:** 23-test suite covering key/address formats, signature verification, PoW boundary conditions, retarget math and clamping, halving purity, double-spend and nonce enforcement, forged-sender rejection, password wrapping, TOTP windows, and a live-node end-to-end scenario (wallet lifecycle, opt-out redirection, cooldowns, transfer, construct+sign, 2FA export, rebind).
- **Operating:** a two-node deployment (author + TLS follower) has synchronized every block since genesis; a five-minute public snapshot and live explorer are published continuously.
- **Open source:** MIT license; no patents, no permissioned forks.

12 • Roadmap

PHASE	SCOPE	STATUS
v1.0 — Genesis	PoW chain, wallets, 2FA, agent mining, follower sync, explorer	Shipped
v1.1 — Federation	Multi-miner block authorship with independent node operators; gossip-based mempool	Design
v1.2 — Light clients	SPV-style headers+merkle proof verification for mobile/web wallets	Design
v2.0 — Governance	On-chain community treasury proposals drawn from fee burns	Research

13 • Conclusion

Jerith Coin demonstrates that a community currency can reward participation without abandoning the auditability of a conventional chain. Its consensus is intentionally orthodox; its issuance is intentionally social. The result is an asset whose distribution follows conversation rather than capital — secured by proof-of-work, verified by anyone, and settled entirely on JerithChain.

14 • References & Resources

- Reference node & full source: github.com/ParadoxFuzz/jeirithchain
- Live explorer: jeirithai.com/explorer.html · [mirror](#)
- Jeirith AI (the agent): jeirithai.com
- Nakamoto, S. *Bitcoin: A Peer-to-Peer Electronic Cash System* (2008) — PoW and retargeting lineage
- RFC 6238 — TOTP: time-based one-time password algorithm (2FA)
- RFC 8032 — Ed25519: high-speed high-security signatures

Disclaimer. This whitepaper is for informational purposes only and does not constitute an offer to sell, a solicitation to buy, or investment advice. Jeirith Coin is a community asset with no promised value, dividend, or return; it may lose value or become illiquid. Digital assets are volatile and may be restricted in your jurisdiction — consult local law before acquiring, mining, or transferring JER. Specifications reflect the reference implementation as of v1.0 (October 2026) and may evolve through the governance process.